Hanxi (Gary) Chen

(201) 406-1327 | hanxic@cs.cornell.edu | www.hanxic.com

EDUCATION

Cornell University — Ithaca, NY
Ph.D. in Computer Science, Programming Languages — Present
Advisor: Prof. Alexandra Silva, Prof. Andrew C. Myers

University of Pennsylvania, *Penn Engineering & The Wharton School* — Philadelphia, PA
M.S.E. in Computer and Information Science — May 2025
B.S.E. in Computer Science, Mathematics minor
B.S. in Economics
Cumulative GPA: 3.97/4.00, *Summa Cum Laude*
Advisor: Prof. Steve Zdancewic, Prof. Sanjeev Khanna

SELECTED PUBLICATIONS

Hanxi Chen, Noam Zilberstein, Andrew C. Myers, Alexandra Silva. Oblivious Probabilistic Outcome Logic: Verifying Probabilistic Programs with an Oblivious Adversary. arXiv, 2026.

Calvin Beck, Hanxi Chen, Steve Zdancewic. Vellvm: Formalizing the Informal. The Eleventh International Workshop on Coq for Programming Languages (CoqPL), 2025.

Calvin Beck, Hanxi Chen, Steve Zdancewic. Vellvm: Formalizing the Informal LLVM (An Experience Report). NASA Formal Methods: 17th International Symposium (NFM), 2025.

Calvin Beck, Irene Yoon, Hanxi Chen, Yannick Zakowski, Steve Zdancewic. A Two-Phase Infinite/Finite Low-Level Memory Model: Reconciling Integer–Pointer Casts, Finite Space, and undef at the LLVM IR Level of Abstraction. Proceedings of the ACM on Programming Languages, 8 (ICFP), 2024.

RELEVANT RESEARCH EXPERIENCE

Research Assistant, *Cornell PL Group*, advised by Prof. Alexandra Silva and Prof. Andrew C. Myers — Sep 2025 – Present
Cornell University, Computer Science

- Developed Oblivious Probabilistic Outcome Logic (opOL), a logic for probabilistic nondeterministic programs under oblivious adversaries, using schedule consumption and probabilistic independence to verify randomized caching and leader-election protocols.
- Mechanized opOL in Lean 4, including monadic semantics, probability-space constructions, probabilistic assertions, semantic and syntactic outcome triples, soundness proofs, and verified case studies.
- Contributed Lean mechanization for Type-Confusion Security, formalizing information-flow checks for decentralized systems where untrusted components can lie about types and induce confused-deputy attacks.

Research Mentor, *Cornell Math+AI*, advised by Prof. Daniel Halpern-Leistner — Jan 2026 – Present
Cornell University, Mathematics

- Mentoring autoformalization for research-level mathematics in Lean 4, curating problem/proof pairs and co-developing proof-quality comparison and LaTeX-to-Lean translation tools.

Research Assistant, *PLClub*, advised by Prof. Steve Zdancewic — Jun 2021 – Aug 2025
University of Pennsylvania, Computer and Information Science

- Contributed to the ICFP 2024 Vellvm memory-model paper in Prof. Steve Zdancewic's group, helping design a low-level memory model for LLVM optimization reasoning, verifying serialization lemmas in Rocq, and implementing tests for the memory model.
- Built Rocq/OCaml validation tools for testing Vellvm's semantic faithfulness against LLVM implementations, including QuickChick generators for UB-free LLVM programs, interpreter-based traces, and a differential testing framework that found novel LLVM bugs.
- Developed Rocq infrastructure for permutation reasoning and list AC unification, then applied it to formalize first-order classical linear logic with cut elimination and a linear-resource calculus for asynchronous queries.

Research Assistant, *Wharton Research Scholars*, advised by Prof. Sanjeev Khanna — Jan 2023 – Dec 2024
University of Pennsylvania, The Wharton School

- Studied precedence-constrained knapsack problems, which are strongly NP-hard in general, and proved pseudo-polynomial algorithms for structured poset classes including unions of three paths and one-directional bipartite posets with consecutive children.
- Presented the work in the Wharton Research Scholars seminar and wrote an undergraduate thesis.

SELECTED TECHNICAL PROJECTS

RISC-V Pipelined Processor – *SystemVerilog, Python, FPGA* — Feb 2025 – May 2025
University of Pennsylvania, *CIS 5710 Computer Organization and Design*

- Implemented a RISC-V pipelined processor with AXI-Lite-style caching, multi-cycle units, branch prediction, and memory-mapped I/O.
- Built verification testbenches, optimized LUT usage and timing behavior, and synthesized the design to FPGA to run compiled programs.

Oat Compiler – *OCaml, C, LLVM IR, x86, Clang, Dune* — Feb 2024 – May 2024
University of Pennsylvania, *CIS 7000 Compilers and Interpreters*

- Implemented a compiler for Oat, a C-like language with pointers and structured data, including lexing, parsing, type checking, frontend lowering, and backend code generation to LLVM IR and x86.
- Added static analyses and optimizations including mem2reg and dead-code elimination, validated against thousands of test cases.

TECHNICAL SKILLS

Proof Assistants: Lean, Rocq/Coq, Mathlib, QuickChick
Programming: OCaml, Haskell, C/C++, Python, SQL, SystemVerilog
Systems and Tools: LLVM IR, Clang, Dune, Git, Unix, LaTeX